

Not All LLM Tasks Are Created Equal: Cost-Effective LLM-Based Fault Localization

Itai Reder
reder@post.bgu.ac.il
Software and Information Systems
Engineering
Ben-Gurion University of the Negev
Beer-Sheva, Israel

Meir Kalech
kalech@bgu.ac.il
Software and Information Systems
Engineering
Ben-Gurion University of the Negev
Beer-Sheva, Israel

Roni Stern
sternron@bgu.ac.il
Software and Information Systems
Engineering
Ben-Gurion University of the Negev
Beer-Sheva, Israel

Abstract

[Placeholder — the abstract of the thesis proposal, carried over verbatim; it does not yet describe this paper.]

Fault localization (FL) with large language models (LLMs) is promising, yet expensive. This proposal studies a hybrid approach that keeps traditional FL to shrink the search space, then uses lightweight LLM reasoning to refine method rankings. The goal is higher accuracy using lower-cost models, while retaining developer-friendly granularity. We frame cost as a first-class metric alongside Top-K, Mean Average Rank (MAR), Mean First Rank (MFR), and Wasted Effort (WE).

In the scope of this study, we will evaluate traditional fault localization methods, LLM-based fault localization methods, and seven different configurations that combine both of these approaches. The first of which is an expensive baseline with strong fault localization accuracy, 3 of which are cheaper baselines following the same methodology, and 3 of which are proof-of-concept hybrid-LLM pipelines that replace some heavy LLM use with targeted calls or smaller models. Experiments use Defects4J [3] at the method level with labels derived from patch edits intersected with failing-test coverage. We expect the two-stage design to preserve most of the accuracy of large LLMs, with clear reductions in price-per-localized-bug. The main contribution of this study is a cost-aware FL framework that utilizes LLMs while balancing accuracy and efficiency.

Keywords

fault localization, large language models, LLM agents, cost-effectiveness, reasoning effort

1 Introduction

[Placeholder — the introduction of the thesis proposal, carried over verbatim; it does not yet describe this paper.]

Motivation. Fault Localization (FL) is a sub-task of software diagnosis, where the goal is to identify the parts in the code responsible for faulty or otherwise unwanted behavior. Developers spend significant time finding *where* a bug lives before they can fix it. Until recently, FL was dominated by statistical methods that analyze test execution data to rank code elements by their likelihood of being faulty. However, the advent of Large Language Models (LLMs) has opened new possibilities for FL, allowing systems to read tests, traces, and code, and propose culprit methods with ease, as well as readable explanations, with little to no effort required on the developer's part. These systems are effective, but the cost of long

prompts and repeated calls limits practical use. For a tool that runs across many projects and versions, the central question becomes: how accurate can we be for a given budget.

Problem statement. We seek a method-level FL pipeline that maintains competitive Top-K, MAR, MFR, and WE while reducing the number, length, or strength of LLM calls. The challenge is to keep enough code context for reliable reasoning without paying for repository-scale prompts or redundant agent steps.

High-level approach. We adopt a two-stage design. Stage 1 performs *space reduction* using traditional FL signals to produce a compact candidate set of suspicious methods. Stage 2 performs *LLM refinement* that inspects only those candidates, using structured prompts and minimal tool calls to score and rank methods. This separation allows us to control where cost is spent and to replace costly steps with cheaper variants without collapsing overall accuracy.

Datasets, granularity, and labeling. We will evaluate on Defects4J at method granularity. Faulty methods are derived by intersecting patch edits with failing-test coverage, using static parsing for method spans and coverage spectra for execution evidence. This yields labels that reflect what developers actually changed and what failing tests executed, which reduces noise and aligns with practical debugging.

Metrics and cost accounting. We report standard FL metrics: Top-K (higher is better), MAR, MFR, and WE (lower is better), and pair these with the average price (\$) for a single a single execution. Token usage and posted model rates determine cost. We macro-average over projects to prevent a few very large or very small cases from dominating results.

Scope and limitations. This is an experimental proposal rather than a final system. Initial runs will use a single pass per version, so small differences between strong models and smaller models may blur due to LLM stochasticity. We address this by macro-averaging, reporting cost side by side with accuracy, and planning expanded repetitions in follow-up experiments.

2 Background and Related Work

[Placeholder — the related-work section of the thesis proposal, carried over verbatim; it does not yet cover the works this paper builds on or position the contribution.]

Fault localization (FL) is crucial for diagnosing software errors by identifying faulty code. Over the years, several approaches have

been developed to optimize this process, ranging from spectrum-based fault localization (SFL) techniques, which utilize statistical analysis of test execution data to rank program elements based on their likelihood of being faulty, to machine learning-based fault localization (MLFL) techniques, which leverage models trained on program features to enhance fault identification. Recently, Large Language Models (LLMs) have shown promise in FL by providing explainability, semantic understanding, and the capacity to process external textual data alongside code and execution data.

2.1 Traditional Fault Localization Methods

2.1.1 Spectrum-Based Fault Localization (SBFL). Spectrum-Based Fault Localization (SBFL) is one of the most widely used techniques for identifying faults in software by leveraging *program spectra*, collected execution data from running a test suite. A *program spectrum* records whether specific software components (e.g., methods, classes) were executed during test runs. By comparing spectra from *passing* and *failing* tests, SBFL assigns *suspiciousness scores* to each component, estimating its likelihood of being faulty.

This approach uses similarity coefficients to rank components, where those executed more often during failures are considered more likely to be faulty. For example, a widely adopted coefficient is the *Ochiai coefficient* [1], defined as:

$$S_{Ochiai} = \frac{n_{11}}{\sqrt{(n_{11} + n_{10})(n_{11} + n_{01})}} \quad (1)$$

Here, n_{11} represents the number of failed tests that executed the component, n_{10} the number of passed tests that did, and n_{01} the number of failed tests that did not.

Although SBFL’s simplicity and effectiveness has kept it a benchmark for comparing newer, more advanced techniques, the *Barinel* algorithm [2] highlights SBFL limitations in complex, multi-fault scenarios, and addresses them by combining spectrum-based observations with Bayesian reasoning for multiple-fault localization. Unlike statistical approaches that rank individual components, Barinel generates multiple-fault diagnosis sets of candidates, and computes their probabilities using a probabilistic component failure model. Two key innovations are the generation of *minimal hitting sets* that explain all failures, and the use of *health factors* $h_j \in [0, 1]$ that represents the probability that a faulty component j produces correct output when executed ($h_j = 0$ means the component consistently causes tests to fail, and $h_j = 1$ means the component is perfectly healthy). This accounts for the realistic scenario where faulty components may pass some tests while failing others. While empirical evidence from both existing literature and our research indicates that Barinel demonstrates little to no performance gains over standard SBFL techniques on the Defects4J benchmark, this observation sheds light on the context-dependent nature of fault localization effectiveness, where optimal methodological approaches may vary significantly across different software projects.

2.1.2 Machine-Learning-Based Fault Localization. Machine learning-based fault localization (MLFL) techniques extend traditional spectrum-based approaches by leveraging statistical models and deep learning architectures to improve fault ranking. These methods often analyze software features beyond execution traces, incorporating structural dependencies and learned representations to provide more accurate

and adaptable fault localization. Early MLFL approaches integrate spectrum, mutation, and textual similarity features using neural networks to enhance fault ranking.

A notable example is *GRACE* [5], a graph-based learning model that treats software coverage as a structured relationship between tests and program components. By representing tests and code as nodes, with execution traces forming edges, GRACE applies *Gated Graph Neural Networks* (GGNNs) to directly learn from this structured data. A key innovation in GRACE is its unified coverage graph representation that integrates fine-grained code structures through statement-level Abstract Syntax Trees (ASTs), where each method is represented as a subgraph of statements and code-block nodes, and test coverage of the method is represented as edges between test nodes and nodes in that method’s subgraph. This allows GRACE to capture syntactic relationships between program elements, such as control flow structures (if statements, loops) and method invocations, while preserving coverage relationships between tests and these fine-grained code components.

Despite their promising results, MLFL methods often rely on labeled training data from previous project versions, and can struggle with generalization across projects. Recent advances, including the use of pre-trained *Large Language Models* (LLMs), aim to address these challenges by incorporating semantic understanding alongside structural analysis, further expanding the capabilities of automated fault localization.

2.2 LLM-Based Fault Localization

The application of Large Language Models (LLMs) to fault localization (FL) represents a shift towards leveraging pre-trained models capable of analyzing source code and identifying potential faults. Unlike traditional fault localization techniques, which rely primarily on statistical correlations or learned representations from execution traces, LLM-based approaches introduce the ability to process software artifacts in a more flexible manner. This has led to increasing interest in their potential to assist with fault localization, particularly in scenarios where structured program analysis alone may be insufficient. Recent studies have explored different ways to incorporate LLMs into the fault localization pipeline, ranging from structured prompting to autonomous and multi-agent frameworks.

2.2.1 Structured Prompting for Fault Localization. An early structured approach to LLM-based fault localization is introduced by Wu et al. [8], who design a systematic method for formatting program data into structured prompts. This approach preprocesses code, test cases, error logs, and execution traces into well-defined templates, which are then fed into an LLM to generate a ranked list of potentially faulty components. By focusing on function-level analysis, this method shows improved accuracy over traditional FL techniques in small to medium-sized projects. However, its effectiveness diminishes at larger scales due to the finite context window of LLMs, making it difficult to process entire classes or projects in a single prompt.

2.2.2 Autonomous LLM Agents. Kang et al. [4] introduce *AutoFL*, an autonomous LLM-based fault localization framework designed to iteratively gather additional context during the fault analysis

process. Unlike static structured prompting methods, AutoFL enables the LLM to dynamically refine its predictions by letting it request snippets and coverage data with iterative function calls. This approach reduces the dependency on fully preprocessed input, addressing some limitations of earlier LLM-based FL techniques.

AutoFL equips the LLM with decision-making capabilities, allowing it to call predefined functions or retrieve additional program data to refine its fault predictions. This adaptability helps mitigate LLM context length constraints, as only relevant information is retrieved on demand.

While improving fault localization accuracy by dynamically refining context, AutoFL introduces additional computational overhead due to multiple LLM invocations. Moreover, its effectiveness relies on well-designed query strategies, as poor query selection may lead to redundant refinement loops, limiting the efficiency of the approach.

Building directly on AutoFL’s foundation, *FlexFL* [9] introduces a practical two-stage hybrid design that addresses some of AutoFL’s computational limitations while maintaining its effectiveness. FlexFL employs a two-phase approach: first, Agent4SR (Space Reduction) combines traditional fault localization techniques with LLM-based global searching to generate a candidate list of suspicious methods; second, Agent4LR (Localization Refinement) performs focused code examination and reasoning on the candidate methods to produce the final ranking. A notable contribution of FlexFL is its demonstration that smaller language models, even open-source ones that can be hosted on end-user machines, can achieve competitive fault localization performance compared to larger proprietary models when combined with traditional FL techniques. This finding is particularly significant for practical deployment, as it reduces both computational costs and dependency on commercial LLM services while still leveraging the semantic understanding capabilities that make LLM-based approaches effective.

2.2.3 Multi-Agent LLM Frameworks. Qin et al. [6] introduce *AgentFL*, a multi-agent fault localization framework that distributes tasks across specialized LLM agents. By assigning different aspects of the fault localization process to individual agents, AgentFL reduces the context length burden on any single agent, mitigating the scalability limitations seen in single-agent approaches.

The framework structures fault localization into a sequential pipeline, where agents specialize in different tasks such as fault comprehension, codebase navigation, and fault confirmation. This structured approach mirrors human debugging workflows, ensuring a systematic analysis of failed test cases. By breaking down the problem into focused steps, AgentFL effectively scales fault localization to larger codebases.

While AgentFL improves scalability and organization, its rigid execution order limits adaptability. Agents follow a predefined workflow, restricting dynamic task reassignment or adjustments based on new information. This may reduce effectiveness in scenarios requiring exploratory or non-linear fault analysis. analysis with complex or unexpected behaviors.

3 Approach

CE-LLM-FL reimplements FlexFL’s two-stage pipeline [9] — Space Reduction (SR) followed by Localization Refinement (LR) — and

decomposes the refinement agent, Agent4LR, into three role-bound slots that can each be assigned a model tier and a reasoning effort independently. This section describes both stages as implemented (Secs. 3.2–3.3), the configuration space over the slots (Sec. 3.4), the prefix-sharing execution machinery that turns slot swaps into paired comparisons (Sec. 3.5), and the fidelity of the reimplementa- tion to the FlexFL release (Sec. 3.6).

3.1 Overview

Figure 1 shows the pipeline for one bug. Stage 1 (SR) reduces the program’s full method corpus to a small ordered candidate set: three classical rankers — Ochiai, SBIR, and BoostNSift (Sec. 2.1.1) — and one LLM agent, Agent4SR, each contribute their top-5 methods, concatenated in the FlexFL release’s fixed order into the candidate set C of exactly 20 entries. Stage 2 (LR) is the role-decomposed Agent4LR: a PLANNER, a TOOL CALLER, and a FINISHER extend one shared conversation transcript in that order, and the finisher’s structured answer — five indices into C — is the final ranking. Each LR slot r carries its own assignment $\sigma_r = (\text{tier}, \text{effort})$; the triple $\sigma = (\sigma_P, \sigma_T, \sigma_F)$ names a configuration (Sec. 3.4).

Crucially, Stage 1 runs *once per bug* with a fixed SR model (llama3.1 : 8b) and its output is shared by every LR configuration: all 39 configurations of the study rank the identical candidate set C from the identical prompt inputs. Every difference we measure between configurations is therefore attributable to the LR slot assignment alone, and every metric in this paper is conditional on C (Sec. 4.4).

3.2 Space Reduction

SR follows the FlexFL release. Its per-bug inputs are the bug report (title and description) and a cleaned trigger test: the failing test method’s source truncated at the failing assertion, a fixed transition sentence, and the stack trace with framework, reflection, and build-tool frames removed. The FlexFL release ships these inputs as pre-curated files; because our benchmarks are rebuilt from scratch, we reimplemented the extraction and validated it against the release’s files (Sec. 3.6).

Classical rankers. Three non-LLM rankers produce method-level suspiciousness lists per bug: Ochiai [1], a spectrum-based ranker over per-test coverage; SBIR, an unsupervised combination of spectrum- and information-retrieval-based ranking (Sec. 2.1.1); and BoostNSift [7], an information-retrieval ranker with query boosting and code sifting. Their configuration as standalone baselines is described in Sec. 4.3; within SR they act only as candidate sources.

Agent4SR. The fourth source is the release’s SR agent: a ReAct-style loop in which the model first writes a free-text plan, then makes up to 10 tool calls, then answers a final prompt with a top-5 list in the release’s `Top_N : (name) template`. Its seven tools — `get_paths`, `get_classes_of_path`, `get_methods_of_class`, `get_code_snippet_of_method`, `find_class`, `find_method`, and `exit` — are all lookups into the two per-bug corpus tables (method signatures and method bodies) that the benchmark preparation emits, not operations on a build tree; the agent is therefore language-agnostic, and the same loop serves Java and Python bugs. Answered method names are resolved against the corpus by the release’s fuzzy

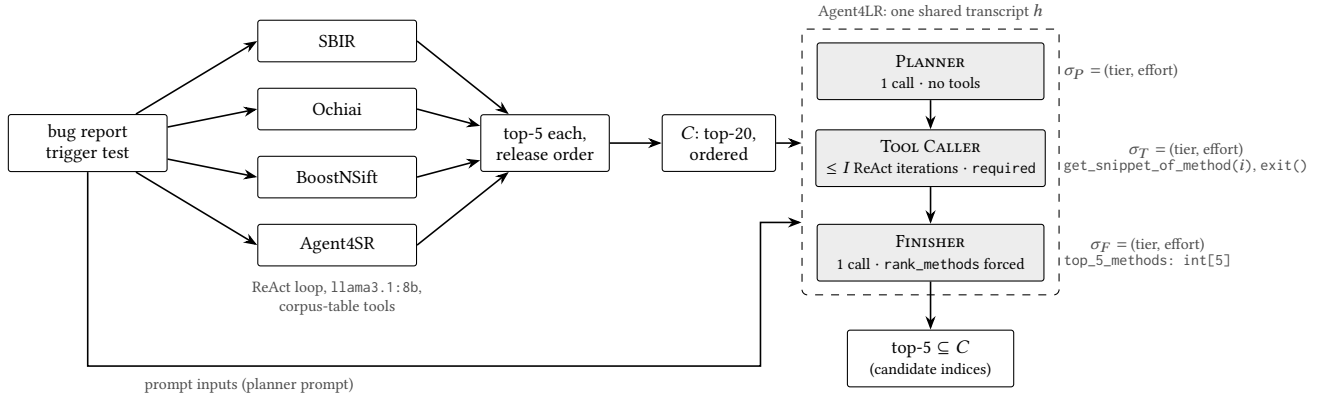


Figure 1: The CE-LLM-FL pipeline for one bug. Stage 1 (Space Reduction) runs once per bug with a fixed model and produces the ordered candidate set C (the SR top-20) shared by every configuration; Stage 2 (Localization Refinement) is the role-decomposed Agent4LR, whose three slots extend one transcript and are independently assigned a model tier and reasoning effort σ_r . The finisher’s forced `rank_methods` call returns indices into C , so no answer-string matching is needed.

matcher, ported line-by-line: token containment first, then Levenshtein distance with the release’s acceptance threshold. Agent4SR runs on `llama3.1:8b` (greedy decoding, temperature 0) via Ollama for every bug in the study; the SR model is never varied.

Combination. The candidate set C is the release’s combination rule verbatim: the top-5 of SBIR, then the top-5 of Ochiai, then the top-5 of BoostNSift, then Agent4SR’s top-5, concatenated in that fixed order with duplicates preserved. A valid C has exactly 20 entries; when a classical source is unavailable, the release fills the classical portion from Ochiai alone (its top-15), whereas we exclude the bug instead so that every evaluated bug faces the same 20-candidate universe (the resulting funnel is quantified in Sec. 4.2). The order of C matters: LR tools address candidates by their 1-based index in this list.

3.3 Role-Decomposed Localization Refinement

The release’s LR stage is a single agent that plans, gathers evidence, and answers inside one conversation. We keep that conversation — the same turn order, the same growing transcript — but split its labor into three role-bound slots at the seams the release’s own prompt structure suggests: a planning turn, a tool-calling loop, and a final-answer turn. Each slot is one *model assignment*; the transcript h is shared state that each slot extends and hands to the next. The transcript is kept in the provider’s native item format (messages, function calls, function-call outputs, and the provider’s reasoning items), so whatever context one slot produces is replayed verbatim to the next, regardless of which model produced it.

PLANNER. The planner makes exactly one LLM call with no tools (`tool_choice=none`). Its input is the seed transcript: a four-sentence system prompt naming the task and the tool-call budget, and a user message containing the bug report, the cleaned trigger test, the numbered candidate list C , and an instruction to reason and plan before calling functions. Its output — a free-text plan — is appended to h . The system prompt’s first sentence (“a debugging

assistant of our Java software”) is inherited verbatim from the release on both benchmarks, including BugsInPy; we verified from the recorded transcripts that Python runs carry this Java wording and note it as a fidelity quirk in Sec. 3.6.

TOOL CALLER. The tool caller runs the evidence-gathering loop: up to $I = 10$ iterations, each consisting of a deterministic reminder (“You have n tool calls remaining”), one LLM call with the tools `get_snippet_of_method(method_number)` and `exit()` under `tool_choice=required` (parallel tool calls disabled), and the local execution of the returned call. `get_snippet_of_method` resolves a 1-based index into C to that method’s source from the corpus tables and appends it to h as a tool observation, in the release’s framing (“The code snippet of `<method>` is as follows.”). A malformed or out-of-range argument appends an error message instead — and, matching the release’s behavior, still consumes an iteration, as does a response carrying no tool call. The loop ends when the model calls `exit()` or the budget is exhausted.

FINISHER. The finisher makes exactly one LLM call. A deterministic handoff message asks for the top-5 most likely culprit methods, and the call offers a single tool, `rank_methods(top_5_methods: int[5])`, under `tool_choice=required` — the model *must* answer with five 1-based candidate indices, ordered most to least likely. The indices resolve exactly to entries of C ; out-of-range indices are dropped (the returned list may then be shorter than five, which the evaluation scores as-is). This is a deliberate departure from the release, which parses free-text “Top_N : `<name>`” answers and resolves them against the corpus by edit-distance matching: forcing a structured index answer removes name-matching noise entirely, which matters here because answer-formatting discipline varies with model tier and would otherwise contaminate the role attribution.

Algorithm 1 gives the full chain; it was verified line-by-line against the runner and its per-call state machine.

Algorithm 1 Role-decomposed Agent4LR chain for one bug

Require: candidate list $C = \langle c_1, \dots, c_{20} \rangle$; inputs X (bug report, trigger test); chain $\sigma = (\sigma_P, \sigma_T, \sigma_F)$ with $\sigma_r = (\text{tier}, \text{effort})$; tool budget I ; tool sets $\mathcal{T}_T = \{\text{get_snippet_of_method}, \text{exit}\}$, $\mathcal{T}_F = \{\text{rank_methods}\}$

Ensure: ranked list of at most 5 methods from C

```

1:  $h \leftarrow \langle \text{Sys}(X, I), \text{PLANPROMPT}(X, C) \rangle$   $\triangleright$  seed transcript
2:  $h \leftarrow h \parallel \text{LLM}_{\sigma_P}(h; \emptyset, \text{choice}=\text{none})$   $\triangleright$  PLANNER
3:  $k \leftarrow 0$ ;  $\text{done} \leftarrow \text{false}$ 
4: while  $\neg \text{done} \wedge k < I$  do
5:    $h \leftarrow h \parallel \text{REMINDER}(I - k)$   $\triangleright$  "You have  $I-k$  tool calls remaining."
6:    $r \leftarrow \text{LLM}_{\sigma_T}(h; \mathcal{T}_T, \text{choice}=\text{required})$   $\triangleright$  TOOL CALLER
7:    $h \leftarrow h \parallel r$   $\triangleright$  response items, incl. reasoning items
8:   for all tool calls  $(f, a) \in r$  do  $\triangleright$  at most one; parallel calls disabled
9:      $h \leftarrow h \parallel \text{EXEC}(f, a)$   $\triangleright$  snippet of  $c_a$ , or error text
10:    if  $f = \text{exit}$  then  $\text{done} \leftarrow \text{true}$ 
11:    end if
12:  end for
13:   $k \leftarrow k + 1$   $\triangleright$  every response consumes an iteration
14: end while
15:  $h \leftarrow h \parallel \text{HANDOFF}()$ 
16:  $r \leftarrow \text{LLM}_{\sigma_F}(h; \mathcal{T}_F, \text{choice}=\text{required})$   $\triangleright$  FINISHER
17:  $\langle i_1, \dots, i_5 \rangle \leftarrow r.\text{top\_5\_methods}$ 
18: return  $\langle c_{i_j} \mid 1 \leq i_j \leq |C| \rangle$   $\triangleright$  out-of-range indices dropped

```

3.4 Configuration Space

Each LR slot is assigned a *model tier* and a *reasoning effort*. The tiers are the three sizes of one provider family — M1 = gpt-5-nano, M2 = gpt-5-mini, M3 = gpt-5 (Table 1) — so a tier step changes capability and price while holding the interface and tokenizer family fixed. The efforts are the Responses API’s per-call reasoning-effort levels: R1 = minimal, R2 = low, R3 = medium, and R4 = high (R4 appears only in a small pilot, Sec. 4.5). A configuration is written P-T-F, naming the planner, tool caller, and finisher assignments in order: M1R1-M1R1-M3R1 runs nano at minimal effort in the first two slots and full gpt-5 at minimal effort as the finisher. The grid of 39 configurations evaluated on both benchmarks is laid out in Sec. 4.5.

Cost is accounted per response. The runner records the provider-reported token usage of every LLM call — input, cached-input, and output tokens (reasoning tokens are billed as output) — and each call is priced with its own model’s list prices from Table 1:

$$\text{cost} = \frac{(t_{\text{in}} - t_{\text{cache}}) p_{\text{in}} + t_{\text{cache}} p_{\text{cache}} + t_{\text{out}} p_{\text{out}}}{10^6}.$$

Because every call is attributed to the slot that issued it, mixed-model chains are priced exactly, and spend can be decomposed by role (Sec. 5). Prices are the provider’s list prices at the time of the experiments.

3.5 Prefix-Sharing Execution

A naive sweep would re-run all three slots for each of the 39 configurations. Instead, the runner writes a content-addressed checkpoint after every completed slot, keyed by a hash of the bug’s input

Table 1: Model tiers and list prices (USD per 1M tokens), as billed by src/evaluation/pricing.py.

Tier	Model	Input	Cached	Output
M1	gpt-5-nano	0.05	0.005	0.40
M2	gpt-5-mini	0.25	0.025	2.00
M3	gpt-5	1.25	0.125	10.00

descriptor (project, bug, benchmark, SR model, candidate source, input keys) together with the identities of the completed slot prefix (role, provider, model, effort, iteration budget per slot). Before executing a configuration, the runner loads the longest stored prefix and re-runs only the diverging suffix.

This is first an efficiency device, but more importantly a design property: two configurations that share a slot prefix share that prefix’s transcript *verbatim*. A finisher swap compares two ranking calls on the identical plan and identical evidence; a tool-caller swap shares the identical plan. Role swaps are therefore paired comparisons in which everything upstream of the swapped slot is held fixed — not merely statistically similar, but byte-identical. Request construction is deterministic in the checkpoint state (the reminder and handoff messages are regenerated, not stored ahead of the call), which also lets an OpenAI Batch workflow drive the same per-call state machine asynchronously for large sweeps; slot-level locks allow concurrent sweeps to share checkpoints without duplicating work.

The gpt-5-family models accept no sampling parameters when a reasoning effort is set — effort is the only decoding knob — and the SR model runs at temperature 0. We execute a single run per (bug, configuration); we do not claim provider-side determinism — the residual nondeterminism is a threat to validity, one that prefix sharing bounds to the swapped suffix.

3.6 Fidelity to the FlexFL Release

Our implementation is a reimplement of the FlexFL release [9], and we anchored it to the release’s observable behavior. Whenever the release provides text, we reuse it verbatim: the SR system prompt and its function-call format instruction, the SR planning instruction, the Top- N answer template, the snippet observation framing, and the four-source combination rule. The release’s monolithic LR prompt was split into the three role prompts of Sec. 3.3, kept as close to the original wording as the decomposition allows. The trigger-test extractor was reverse-engineered against the release’s shipped input files using paired fixtures across five Defects4J projects. Every behavioral deviation we are aware of is enumerated in Table 2; remaining differences are at the whitespace level. The two consequential deviations are structured tool calling and the forced rank_methods answer, which together replace the release’s free-text function-call parsing and Levenshtein-based answer resolution — machinery whose failure rate varies with model tier and would otherwise be confounded with the role effects this paper measures.

Table 2: Fidelity to the FlexFL release: every known behavioral deviation of our reimplementation. Prompt text not listed here is reused verbatim from the release.

Aspect	FlexFL release	This work
LR agent structure	One agent, one model: plan, tool calls, and answer in a single conversation	Same conversation and turn order, split into three role-bound slots (PLANNER, TOOL CALLER, FINISHER), each with its own (tier, effort) assignment
Tool invocation	Free-text <code>FunctionName(Argument)</code> lines parsed from the completion; malformed lines re-prompted with the function list	Native structured tool calls with per-slot tool sets and <code>tool_choice</code> (none/required/required); parallel calls disabled; a malformed argument yields an error observation and still consumes budget; SR keeps a one-line retry nudge (reworded)
Function list	Inlined in the system prompt with per-argument explanations	Carried as structured tool schemas; descriptions shortened and reworded
Per-iteration prompt (LR)	“Now call a function in this format ‘ <code>FunctionName(Argument)</code> ’ in a single line without any other word.”	Deterministic budget reminder: “You have n tool calls remaining.”
LR final answer	Free-text “ <code>Top_N : <name></code> ” lines, resolved against the corpus by token containment, then Levenshtein matching	Forced <code>rank_methods(top_5_methods: int[5])</code> ; 1-based indices into <code>C</code> resolve exactly; out-of-range indices dropped
LR snippet tool	<code>get_code_snippet_of_method(method_number)</code>	Renamed <code>get_snippet_of_method(method_number)</code> ; same 1-based index semantics and observation framing
LR prompt wording	Monolithic system prompt; candidate header “The suggested methods are as follows”; planning instruction “Let’s locate the faulty method step by step...”	System prompt reworded into four sentences; candidate header “The most likely culprit methods are”; planning instruction “Before you start calling functions, reason and plan your approach...”
Input preparation	Bug reports and trigger tests shipped as pre-curated files	Extractor reverse-engineered against the release’s files (test source truncated at the failing assertion + cleaned stack trace); Jira markup stripped at LR prompt-build time only
Missing candidate source	Classical portion filled from Ochiai alone (top-15)	Bug excluded unless the combined list has exactly 20 entries (fixed candidate universe; Sec. 4.2)
Models	Meta-Llama-3.1-8B-Instruct hosted locally, both stages	SR fixed at Llama3.1:8b via Ollama; LR slots drawn from the gpt-5 family via the Responses API
Benchmarks & language	Defects4J and GHRB (Java)	Defects4J and BugsInPy; the SR prompt adapts the language noun and answer template for Python; the LR system prompt inherits “Java software” verbatim on both benchmarks

4 Experimental Setup

This section states the six research questions (Sec. 4.1), the benchmarks and the funnel from benchmark bug to evaluated panel (Sec. 4.2), the classical baselines (Sec. 4.3), the metrics and the fixed bug panel every number in this paper is computed on (Sec. 4.4), the configuration grid (Sec. 4.5), and the implementation and reproducibility of the study (Sec. 4.6).

4.1 Research Questions

All six questions concern Stage 2: which per-role assignments of model tier and reasoning effort buy fault-localization accuracy, and which only buy tokens. Accuracy is read on MAR (mean average rank of the faulty methods; lower is better) and cost on mean USD per bug, both defined in Sec. 4.4. Throughout, m_i abbreviates the tier- i model at minimal reasoning effort (MiR1), $U(m)$ is the uniform configuration m - m - m , and $X(m_i, m)$ with $X \in \{P, T, F\}$ is the uniform background $U(m_i)$ with role X swapped to variant m ; Sec. 4.5 enumerates the grid this notation spans.

RQ1 (role $\rightarrow$ accuracy). *Which Agent4LR role affects FL accuracy the most?* For each role X and each background tier i , the sweep $X(m_i, \cdot)$ moves one role through the five swept variants while the other two roles stay fixed; the spread (Max–Min) of MAR across a sweep measures how sensitive accuracy is to that role.

RQ2 (role $\rightarrow$ cost). *Which Agent4LR role affects the monetary cost of a run the most?* The same sweeps, read on mean cost per bug — reported in raw USD and min–max normalized over each benchmark’s own grid cost range, so the two benchmarks’ spreads are comparable.

RQ3 (reasoning effort). *Does a higher reasoning effort yield better FL accuracy?* The uniform reasoning axis $U(M2R1) \rightarrow U(M2R2) \rightarrow U(M2R3)$ — the whole agent’s effort raised one step at a time — with paired per-bug bootstrap confidence intervals on each step (Sec. 4.4).

RQ4 (model tier). *Does a higher-tier model yield better FL accuracy?* The uniform tier axis $U(m_1) \rightarrow U(m_2) \rightarrow U(m_3)$, which shares its anchor $U(m_2) = U(M2R1)$ with RQ3’s axis so the two upgrade paths are directly comparable.

RQ5 (price of a step). *By how much does a step up in reasoning effort, versus a step up in model tier, increase the cost of a run?* Cost multipliers of consecutive steps along the two uniform axes of RQ3/RQ4, and the median per-role multipliers when only a single role takes the step.

RQ6 (mixed vs. uniform). *Does assigning different models to different roles yield a better cost–accuracy trade-off than a single homogeneous agent?* All 39 configurations are placed in the cost/MAR plane and measured against the frontier spanned by the uniform configurations, counting as decisive only the mixed configurations that beat the frontier by more than a fixed relative margin on both axes.

4.2 Benchmarks and the Bug Funnel

The study runs on two method-level FL benchmarks: Defects4J (Java; 854 bugs across 17 projects) and BugsInPy (Python; 501 bugs across 17 projects). Both are rebuilt from scratch by our extraction pipeline (Sec. 3.6): each bug is checked out and compiled in a per-benchmark container, per-test coverage is collected (GZoltar on Defects4J, FauxPy on BugsInPy), the bug report is fetched, and the ground truth $\mathcal{F}$ is the set of methods modified by the developer

patch. Not every benchmark bug is evaluable at every stage, and the denominators differ by stage, so we state the funnel once and exactly.

Defects4J. All 854 bugs complete extraction and classical FL, so the three baselines are evaluated on the full benchmark. The faulty methods intersect the candidate set C for 616 bugs (72.1% of the benchmark); for the remaining bugs every ranking of C is vacuous and all rank metrics are blank (Sec. 4.4). The grid produced rankings for 626 bugs, of which 10 are bugs whose faults lie outside C . The full-sweep panel (Sec. 4.4) therefore holds 616 bugs, and on Defects4J it coincides exactly with the set of grid-run bugs whose fault appears in C : every such bug completed all 39 configurations, so no bug is lost to an incomplete sweep.

BugsInPy. Of the 501 bugs, 213 are not fully evaluable; a committed exclusion table records one reason per bug, in seven buckets: the extraction audit could not validate the reproduced test failure (21 bugs), the faulty methods are never touched by the coverage spectra (19), the developer patch modifies no function body, leaving no method-level ground truth (13), SR never ran because no trigger-test failure could be mirrored (23), SR produced no valid 20-entry candidate list (47; this includes the missing-source exclusion rule of Sec. 3.2), the fault lies outside the SR top-20 (71), and the LR stage was not run (19). The classical baselines are evaluated for 390 bugs; the fault intersects C for 307 of them (78.7%); and the grid completed all 39 configurations on 288 of those — the 19 LR-not-run bugs account exactly for the difference.¹

4.3 Baselines

The three classical rankers that serve as SR candidate sources (Sec. 3.2) are also evaluated standalone as baselines. Ochiai scores every statement with the Ochiai coefficient over the per-test coverage spectra and lifts statement scores to methods by taking each method’s maximum. SBIR combines the statement-level spectrum scores with Blues, an information-retrieval ranker over the bug report, by unsupervised rank aggregation (RAFL) following the SBIR recipe (Sec. 2.1.1), lifted to methods the same way. BoostNSift is a method-level information-retrieval ranker with query boosting and code sifting. Each baseline’s method scores are restricted to the candidate set C , ranked by the procedure of Sec. 4.4, and aggregated over the same bug panels as the Agent4LR configurations — every baseline-versus-Agent4LR comparison shares both its candidate universe and its denominator.

Two comparisons are deliberately absent. We do not compare against the accuracy numbers published for FlexFL, AutoFL, or any other end-to-end system: our metrics are conditional on C and on the panel (Sec. 4.4), theirs are absolute over whole benchmarks, and the two are not on a common scale. And we do not re-run the FlexFL release end-to-end as a baseline: the object of study is the LR slot assignment, so SR is held fixed and identical across all arms (Sec. 3.1), whereas an end-to-end re-run would vary exactly the stages this design controls for. Every claim below is therefore

¹The committed cross-bug summary CSVs average the BugsInPy baselines over the 307 fault-in- C bugs; every baseline number in this paper is instead recomputed on the 288-bug panel (Sec. 4.4), so baselines and Agent4LR configurations always share a denominator.

about slot assignment given that fixed SR stage, and not about the pipeline as a whole.

4.4 Metrics and the Full-Sweep Panel

Rank assignment. Every FL method — classical or LLM — is scored identically. The bug’s candidate universe is the SR top-20 C ; the method’s scores over C become 1-based ranks; tied scores share the mean of their positions (with a deterministic secondary order on corpus identity); and candidates the method leaves unscored are appended with one uniform tail rank, the mean of the remaining positions. An Agent4LR configuration ranks by the order of its returned list (at most five methods, Sec. 3.3), so its unreturned candidates share the tail rank; a baseline’s suspiciousness scores rank all of C . A bug therefore yields a rank for every method whenever $\mathcal{F}_C = \mathcal{F} \cap C$ is non-empty; when $\mathcal{F}_C = \emptyset$ all rank metrics are blank.

Metrics. Per bug: FR (first rank) is the lowest rank of any method in $\mathcal{F}_C$; AR (average rank) the mean rank over $\mathcal{F}_C$; Top- k , $k \in \{1, \dots, 5\}$, the indicator $[\text{FR} \leq k]$; and wasted effort $\text{WE} = (r_{\text{last}} - |\mathcal{F}_C|) / (|C| - |\mathcal{F}_C|)$, where r_{last} is the worst rank in $\mathcal{F}_C$ — the fraction of healthy candidates inspected before every fault is found. Cost per bug is the sum of the per-call USD amounts of Sec. 3.4, with input, cached, and output token counts kept alongside. Across a panel we report the means MFR, MAR, Top- k rate, mean WE, and mean USD per bug; MAR and mean USD per bug are the primary axes of every research question, while Top- k , MFR, and WE appear where the baselines are introduced and in the appendix. Where a comparison between two configurations carries uncertainty, we use a paired per-bug bootstrap: the per-bug AR difference on the shared panel is resampled over bugs (2000 resamples) and reported as the mean difference with its 95% percentile interval.

The full-sweep panel. A bug enters a benchmark’s panel iff every one of the 39 grid configurations has a non-blank FR for it — the bug’s fault lies in C and the entire sweep completed. This yields the 616 Defects4J and 288 BugsInPy bugs of Sec. 4.2, 904 in total. All aggregates of all methods — grid configurations, off-grid uniforms (Sec. 4.5), and classical baselines — are computed over these fixed panels, so on a given benchmark every mean in the paper has the same denominator.

Conditionality. Every metric in this paper is thus conditional twice over: on the SR stage — each number measures ranking quality *within* the fixed candidate set C , not end-to-end localization over the whole program — and on the full-sweep panel. The results captions restate this condition, and no number here is comparable to absolute figures reported for end-to-end systems (Sec. 4.3).

4.5 Configuration Grid

The grid instantiates the assignment space of Sec. 3.4 as a swap design around three homogeneous anchors. The anchors are the uniform configurations $U(m_i) = \text{MiR1-MiR1-MiR1}$, $i \in \{1, 2, 3\}$. For each anchor and each role $X \in \{P, T, F\}$, the role is swapped to one of the five variants $m \in \{\text{M1R1}, \text{M2R1}, \text{M2R2}, \text{M2R3}, \text{M3R1}\}$: only the mid tier M2 sweeps its reasoning range R1–R3, while M1 and M3 stay pinned at R1 to bound the grid. The resulting 45 sweep instances collapse to 39 unique configurations, because the three anchors are shared across the roles’ sweeps. Within a sweep

$X(m_i, \cdot)$ everything outside the swapped role is fixed — and, by prefix sharing (Sec. 3.5), byte-identical — so each sweep reads as a paired comparison.

The grid’s uniforms sit at minimal effort only, but the reasoning axis of RQ3/RQ5 needs uniform chains at higher effort. Both benchmarks therefore also ran the off-grid uniforms $U(M2R2)$ and $U(M2R3)$, read on the same panels: on Defects4J, $U(M2R2)$ covers 616 of the 616 panel bugs and $U(M2R3)$ covers 615 (the one missing bug is noted in every caption it appears in); on BugsInPy both cover all 288. They enter RQ3, RQ5, and RQ6 as uniform points and never enter the role sweeps.²

Everything else is fixed across the study: the tool budget $I = 10$, the SR stage (one llama3.1 : 8b run per bug at temperature 0, shared by all configurations), and a single LR run per (bug, configuration) pair (Sec. 3.5).

4.6 Implementation and Reproducibility

The pipeline is implemented in Python. LR slots call the OpenAI Responses API, with the provider’s reasoning items carried across slot handoffs in the shared transcript (Sec. 3.3); large sweeps drive the identical per-call state machine through the OpenAI Batch API (Sec. 3.5); Agent4SR calls a locally hosted llama3.1 : 8b through Ollama. Token counts are the provider-reported usage fields of each response, priced at the list prices of Table 1.

Reproducibility rests on a committed, self-verifying data layer. The repository carries, for every evaluated bug, the exact input closure of every metric: the candidate universe C , each ranker’s method-level scores, the ground-truth faulty methods, and each configuration’s returned ranking with its per-call token usage. The builder that distills this layer from the raw processed tree refuses to write a bug unless the metrics recomputed from the slim copy equal those computed from the full tree. Re-running the evaluator on an unmodified checkout therefore rewrites every per-bug and cross-bug metric table byte-identically — the committed CSVs are a fixed point of the evaluation code, not a snapshot to be trusted — and needs no benchmark checkout, no containers, and no API keys. Every number, table, and figure in this paper is generated from those committed tables; none is hand-typed. The artifact is available at https://github.com/itai-reder/CE_LLM_FL.

5 Results

Every research question is answered on the same plane: accuracy as MAR and spend as mean USD per bug, computed per benchmark on the fixed full-sweep panels of Sec. 4.4 (616 Defects4J and 288 BugsInPy bugs) and conditional throughout on the shared SR top-20 candidate set C . RQ1 and RQ2 traverse this plane along the role sweeps — one slot moved, everything else byte-identical — reading the vertical (MAR) and horizontal (cost) extent of each sweep respectively; RQ3–RQ5 move along the two uniform axes through $U(M2R1)$; RQ6 reads the plane as a whole, mixed configurations against the uniform frontier. Table 3 first anchors the absolute level of the plane: the three classical rankers, rescored inside C on the panel denominators, against the three uniform Agent4LR chains.

²Beyond the grid, the committed evaluation tables retain exploratory runs that no table or figure in this paper uses: a uniform pilot of the R4 (high) effort on 9 Defects4J bugs, single-bug probes of off-grid (tier, effort) pairs, and probes of a non-OpenAI (qwen-family) model in the tool-caller and finisher slots.

Every uniform chain outranks every classical baseline on every metric on both benchmarks — the all-nano chain $U(m_1)$ does so for a fraction of a cent per bug — so the questions that follow are not whether the LR stage helps, but which of its slots the improvement should be bought for. Top- k , MFR, and WE appear in this table only; the remainder of the section reads MAR and cost, the primary axes.

5.1 RQ1: Which Role Affects Accuracy the Most?

Figure 2 lays out all role sweeps on both benchmarks: one panel per (role, benchmark) pair, and within each panel one series per uniform background $U(m_i)$, whose points are that background’s five swap variants labeled by the swapped assignment. Because of prefix sharing (Sec. 3.5), each series is a family of paired comparisons: every MAR difference within a series is attributable to the swapped slot alone. Table 4 condenses each of the eighteen sweeps to its Max–Min spread — the distance between the sweep’s best and worst value — which we read on the MAR columns here and on the cost columns in RQ2.

The FINISHER panels are unlike the other four. Their sweeps are near-vertical: moving the FINISHER assignment sweeps MAR through a Max–Min range of 1.422–1.641 on Defects4J and 1.197–1.293 on BugsInPy, across all three backgrounds, while barely moving cost. No other role comes close: the widest PLANNER spread is 0.269 (Defects4J) and 0.409 (BugsInPy), the widest TOOL CALLER spread 0.554 and 0.303. Even the narrowest FINISHER spread exceeds the widest spread of any other role by a factor of 2.56 on Defects4J and 2.93 on BugsInPy.

The paired swaps between the extreme backgrounds make the asymmetry concrete. Upgrading only the FINISHER of the all-nano chain to the top tier — M1R1–M1R1–M3R1, the point labeled m3 on the $U(m_1)$ series of the FINISHER panels — lowers MAR from 3.963 to 2.322 on Defects4J, closing 86.0% of the entire $U(m_1) \rightarrow U(m_3)$ accuracy gap with two of three slots still on the cheapest assignment; on BugsInPy the same swap closes 96.0% of the gap (MAR 2.757 to 1.541). The converse swap forfeits what the upgrade bought: downgrading only the FINISHER of the all-gpt-5 chain to nano yields MAR 3.628 on Defects4J — 82.4% of the gap surrendered — and 2.783 on BugsInPy, 102.1% of the gap: slightly worse than the all-nano chain itself, despite the PLANNER and TOOL CALLER both running the top tier.

The PLANNER and TOOL CALLER, by contrast, move MAR little, and neither consistently outranks the other. On Defects4J the TOOL CALLER spreads (0.195–0.554) sit above the PLANNER spreads (0.090–0.269); on BugsInPy the widest non-FINISHER spread belongs to the PLANNER on the $U(m_1)$ background (0.409 vs. 0.303 for the TOOL CALLER), and both roles’ spreads shrink well below the FINISHER’s on the stronger backgrounds (Table 4). What their sweeps chiefly move is cost — the horizontal extent of the PLANNER and TOOL CALLER panels in Fig. 2 — which is RQ2’s subject.

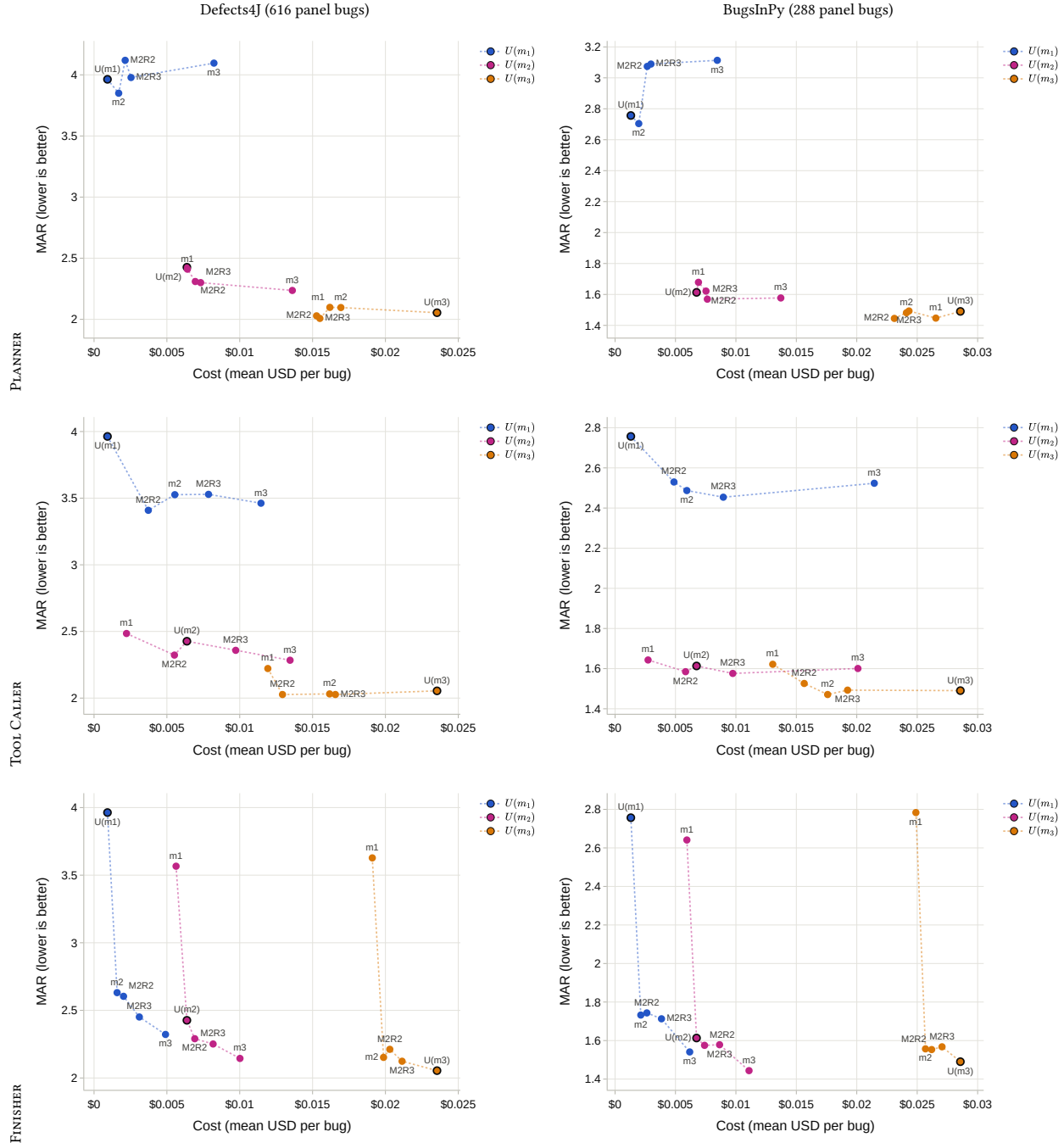


Figure 2: The role sweeps in the cost/MAR plane (RQ1/RQ2): one row per swapped role, one column per benchmark. Within a panel, each series holds a uniform background $U(m_i)$ fixed (black-ringed marker) and reassigns only that role; points are labeled by the swapped assignment (m1, m2, m3 = the tier at minimal effort; M2R2, M2R3 = the mid tier at higher effort). Lower left is better. By prefix sharing, every within-series difference is attributable to the swapped slot alone. MAR and mean cost are computed on the full-sweep panels and are conditional on the shared SR top-20 candidate set C (Sec. 4.4).

Table 3: Classical baselines vs. the three uniform Agent4LR chains. All metrics are conditional on the shared SR top-20 candidate set C and are computed on the full-sweep panels (Sec. 4.4), so baselines and chains share their denominators; the baselines are rescored on these panels rather than copied from the benchmark-wide summaries. Lower is better for MAR, MFR, and WE; higher for Top- k ; classical rankers incur no LLM cost.

Method	Defects4J (616 bugs)						BugsInPy (288 bugs)					
	MAR	MFR	Top-1	Top-5	WE	\$/bug	MAR	MFR	Top-1	Top-5	WE	\$/bug
Ochiai	4.703	4.211	0.281	0.713	0.233	–	3.471	3.170	0.253	0.802	0.203	–
SBIR	5.220	4.735	0.315	0.615	0.268	–	4.127	3.837	0.312	0.660	0.247	–
BoostNSift	6.461	6.118	0.201	0.442	0.329	–	5.507	5.222	0.181	0.455	0.356	–
$U(m_1)$ M1R1–M1R1–M1R1	3.963	3.307	0.477	0.854	0.198	0.0009	2.757	2.491	0.514	0.906	0.139	0.0013
$U(m_2)$ M2R1–M2R1–M2R1	2.426	1.964	0.705	0.953	0.098	0.0064	1.613	1.385	0.830	0.990	0.053	0.0067
$U(m_3)$ M3R1–M3R1–M3R1	2.054	1.666	0.794	0.969	0.072	0.0235	1.490	1.274	0.844	0.993	0.044	0.0286

Table 4: Max–Min spread of each role’s five-variant sweep on each uniform background: read on MAR for RQ1 and on min–max-normalized mean cost per bug for RQ2. All values are computed on the full-sweep panels and are conditional on the shared SR top-20 candidate set C (Sec. 4.4).

Background	Role	Max–Min MAR		Max–Min cost (norm.)	
		D4J	BIP	D4J	BIP
$U(m_1)$	Planner	0.269	0.409	0.323	0.262
	Tool Caller	0.554	0.303	0.466	0.739
	Finisher	1.641	1.216	0.176	0.179
$U(m_2)$	Planner	0.190	0.109	0.320	0.256
	Tool Caller	0.201	0.067	0.496	0.637
	Finisher	1.422	1.197	0.194	0.189
$U(m_3)$	Planner	0.090	0.048	0.365	0.200
	Tool Caller	0.195	0.151	0.513	0.569
	Finisher	1.573	1.293	0.196	0.135

Answer to RQ1

The FINISHER, decisively, on both benchmarks: its sweeps span a Max–Min MAR of 1.422–1.641 (Defects4J) and 1.197–1.293 (BugsInPy), at least $\times 2.56$ and $\times 2.93$ the widest spread of any other role. Upgrading the finisher alone closes 86.0% (Defects4J) and 96.0% (BugsInPy) of the $U(m_1) \rightarrow U(m_3)$ accuracy gap, while downgrading it alone surrenders 82.4% and 102.1% of that gap.

5.2 RQ2: Which Role Affects Cost the Most?

Read on the horizontal axis, Fig. 2 inverts RQ1’s picture: the FINISHER sweeps that decided MAR are near-vertical, while the TOOL CALLER sweeps stretch farther along the cost axis than those of any other role, on every background and both benchmarks. The normalized-cost columns of Table 4 quantify this. Because each benchmark’s grid cost range is spanned by its two extreme uniform chains — $U(m_1)$ at \$0.0009 and $U(m_3)$ at \$0.0235 mean cost per bug on Defects4J, \$0.0013 and \$0.0286 on BugsInPy — a normalized spread reads directly as the fraction of the all-nano-to-all-gpt-5 cost gap that moving one role traverses. The TOOL CALLER sweeps

traverse 0.466–0.513 of that gap on Defects4J and 0.569–0.739 on BugsInPy — raw Max–Min swings of \$0.0105–\$0.0116 and \$0.0155–\$0.0201 per bug. The PLANNER sweeps stay at 0.320–0.365 (Defects4J) and 0.200–0.262 (BugsInPy), and the FINISHER sweeps — the role that decided RQ1 — are the narrowest of all: 0.176–0.196 and 0.135–0.189. The ordering TOOL CALLER > PLANNER > FINISHER holds on every background of both benchmarks (Table 4).

The paired swaps that anchored RQ1 make the asymmetry concrete in dollars. Upgrading only the TOOL CALLER of the all-nano chain to the top tier (M1R1–M3R1–M1R1) multiplies the whole chain’s mean cost by $\times 12.43$ on Defects4J and $\times 16.41$ on BugsInPy — from \$0.0009 to \$0.0115 and from \$0.0013 to \$0.0214 per bug, i.e., 46.6% and 73.9% of the uniform cost gap — while RQ1 bounds the MAR movement of any TOOL CALLER sweep at 0.554 (Defects4J) and 0.303 (BugsInPy). The FINISHER swap of RQ1, which closed 86.0% and 96.0% of the accuracy gap, incurs a fraction of that spend: it moves the chain only to \$0.0049 and \$0.0062 per bug — 17.6% and 17.9% of the same gap.

The TOOL CALLER’s dominance is structural: it is the only slot that pays per iteration. It issues up to I calls per bug (Sec. 3.3), each billed at its own assignment’s prices (Sec. 3.4), and each call re-reads the transcript accumulated so far — a transcript that the snippet observations it fetches keep growing — so its reasoning and output tokens are spent up to I times per bug, and its assignment multiplies through the loop rather than through a single completion. The PLANNER and FINISHER each make exactly one call, so swapping either re-prices a single completion. Why the PLANNER spreads nonetheless sit consistently above the FINISHER’s, we can only conjecture: the planner both emits a longer free-text completion (a plan rather than five indices) and sits first in the chain, where its plan shapes what every later call reads and does, whereas the finisher re-prices nothing downstream — but our accounting attributes each call’s spend to the slot that issued it, not to the slots whose output it reads, so we do not separate these two effects here.

Together, RQ1 and RQ2 expose the asymmetry the remaining questions price out: the accuracy-critical role is cheap to upgrade — 86.0%/96.0% of the accuracy gap for 17.6%/17.9% of the cost gap — while the cost-critical role buys almost no accuracy. RQ5 prices single upgrade steps against this backdrop, and RQ6 asks whether mixed chains that exploit the asymmetry — frugal PLANNER and TOOL CALLER, stronger FINISHER — beat the uniform frontier.

Answer to RQ2

The TOOL CALLER, on both benchmarks: its sweeps traverse 0.466–0.513 (Defects4J) and 0.569–0.739 (BugsInPy) of the $U(m_1) \rightarrow U(m_3)$ cost gap, versus at most 0.365 / 0.262 for the PLANNER and 0.196 / 0.189 for the FINISHER. Upgrading the TOOL CALLER alone multiplies the all-nano chain’s cost by $\times 12.43$ (Defects4J) and $\times 16.41$ (BugsInPy), while the accuracy-decisive FINISHER upgrade of RQ1 costs only 17.6% and 17.9% of the same gap.

5.3 RQ3: Does Higher Reasoning Effort Improve Accuracy?

RQ3 and RQ4 leave the role sweeps and read the two uniform axes of Sec. 4.5: chains that give every slot the same assignment, so that each step upgrades the whole agent at once rather than a single role — the per-role reading was RQ1’s subject and is not repeated here. Figure 3 places the five uniform chains in the cost/MAR plane; Table 5 reads both axes step by step, attaching to each consecutive step the mean per-bug AR change with its paired per-bug bootstrap interval (Sec. 4.4; negative favors the upgrade). RQ3 walks the reasoning axis $U(M2R1) \rightarrow U(M2R2) \rightarrow U(M2R3)$: the mid-tier model in every slot, with the whole agent’s reasoning effort raised from minimal through low to medium.

On Defects4J the axis moves in the right direction, but only its second step carries evidence. Raising the whole agent from minimal to low effort moves MAR from 2.426 to 2.303 — a paired change of $-0.124 [-0.281, +0.040]$, whose interval straddles zero — while raising it from low to medium moves MAR on to 2.107, a change of $-0.196 [-0.356, -0.036]$, whose interval excludes zero. What gain the axis delivers is therefore concentrated in its R2→R3 step — exactly the step that RQ5 prices at $\times 2.08$ the chain’s cost, nearly the whole $\times 2.11$ the axis compounds to.

On BugsInPy the axis is flat. MAR moves from 1.613 through 1.594 to 1.573 — paired changes of $-0.020 [-0.139, +0.107]$ and $-0.021 [-0.139, +0.092]$, both intervals comfortably straddling zero. We do not average the two benchmarks into a single verdict: on BugsInPy, raising the whole agent’s reasoning effort bought no measurable accuracy at any step. Figure 3 suggests how little room the axis has to work with there: its anchor $U(M2R1)$ already sits at MAR 1.613, close to the all-gpt-5 chain’s 1.490, so most of the movement available within C was already collected by the tier step into the anchor (RQ4, Sec. 5.4).

Even where the axis does move, it moves modestly by the standard of the tier axis it shares its anchor with: the full two-step walk on Defects4J ends at MAR 2.107, still short of the $U(m_3)$ chain’s 2.054, and either single tier step of Table 5 shifts MAR by more than both reasoning steps together. RQ4 reads that axis next; RQ5 then prices the two kinds of step against each other.

Answer to RQ3

Modestly on Defects4J, and only at the top of the range: the R2→R3 step is significant (paired AR change $-0.196 [-0.356, -0.036]$) but the R1→R2 step is not ($-0.124 [-0.281, +0.040]$). On BugsInPy, no: MAR barely moves (1.613 → 1.573) and neither step’s interval excludes zero ($-0.020 [-0.139, +0.107]$, $-0.021 [-0.139, +0.092]$).

5.4 RQ4: Does a Higher Model Tier Improve Accuracy?

RQ4 walks the second uniform axis of Figure 3, $U(m_1) \rightarrow U(m_2) \rightarrow U(m_3)$: every slot at minimal reasoning effort, with the whole agent’s model tier raised from gpt-5-nano through gpt-5-mini to gpt-5. The tier rows of Table 5 read the axis step by step; as in RQ3, the per-role tier sweeps are not reread here.

The first step is large and unambiguous on both benchmarks. On Defects4J, raising every slot from m_1 to m_2 moves MAR from 3.963 to 2.426 — a paired per-bug AR change of $-1.537 [-1.826, -1.254]$, the largest movement any single step on either uniform axis produces, with an interval far from zero. On BugsInPy the step moves MAR from 2.757 to 1.613, a change of $-1.143 [-1.432, -0.870]$, equally clearly significant.

The second step splits the benchmarks. On Defects4J, raising every slot on from m_2 to m_3 still buys a significant improvement: MAR moves on to 2.054, a paired change of $-0.372 [-0.538, -0.210]$, whose interval excludes zero — about a quarter of the first step’s movement, but real. On BugsInPy the same step moves MAR only from 1.613 to 1.490, a change of $-0.123 [-0.266, +0.010]$; the interval reaches past zero, and we do not read the step as significant there. This is the pattern RQ3 anticipated: on BugsInPy the $m_1 \rightarrow m_2$ step collects most of the movement available within the candidate set C , leaving neither the reasoning axis (RQ3) nor the top tier step measurable room to improve on the $U(m_2)$ anchor.

Set against RQ3, the tier axis is the one that moves the needle. The two axes share their anchor $U(m_2) = U(M2R1)$, so Table 5 compares them directly: on Defects4J the $m_1 \rightarrow m_2$ tier step alone ($-1.537 [-1.826, -1.254]$) shifts mean AR by several times both reasoning steps combined ($-0.124 [-0.281, +0.040]$ and $-0.196 [-0.356, -0.036]$), and even the disputed $m_2 \rightarrow m_3$ step ($-0.372 [-0.538, -0.210]$) exceeds either of them. Tier, not reasoning effort, is where the uniform axes concentrate their accuracy — and, as RQ5 shows next, also where they concentrate their cost.

Answer to RQ4

Yes at the bottom of the range, and only partly at the top. The $m_1 \rightarrow m_2$ step is large and significant on both benchmarks (paired AR change $-1.537 [-1.826, -1.254]$ on Defects4J, $-1.143 [-1.432, -0.870]$ on BugsInPy). The $m_2 \rightarrow m_3$ step is significant on Defects4J ($-0.372 [-0.538, -0.210]$) but not on BugsInPy ($-0.123 [-0.266, +0.010]$), where the first step already collects most of the movement available within C .

5.5 RQ5: What Does a Step Cost?

RQ3 and RQ4 read the two uniform axes of Figure 3 on accuracy; RQ5 reads the same two axes on cost, and asks what a single upgrade

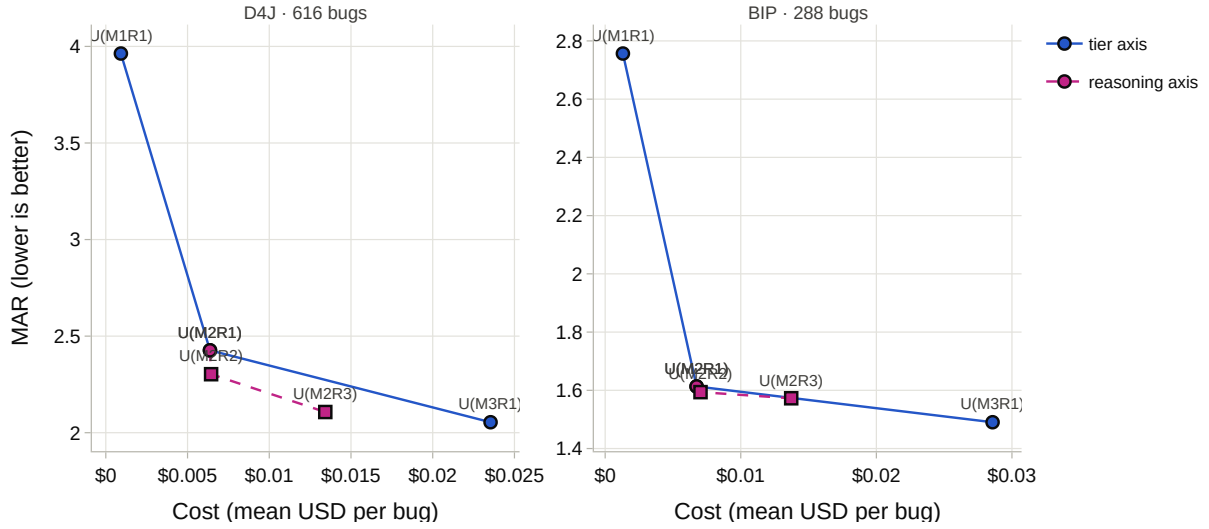


Figure 3: The five uniform chains in the cost/MAR plane: the tier axis $U(m_1) \rightarrow U(m_2) \rightarrow U(m_3)$ (RQ4) and the reasoning axis $U(M2R1) \rightarrow U(M2R2) \rightarrow U(M2R3)$ (RQ3), sharing the anchor $U(m_2) = U(M2R1)$; RQ5 reads the same two axes on cost. Lower left is better. MAR and mean cost per bug are computed on the full-sweep panels and are conditional on the shared SR top-20 candidate set C (Sec. 4.4); on Defects4J, $U(M2R3)$ covers 615 of the 616 panel bugs (Sec. 4.5).

Table 5: The two uniform axes read step by step (RQ3: reasoning; RQ4: tier). Each axis lists MAR at its three steps and, for each consecutive step, the mean per-bug AR change with its paired per-bug bootstrap percentile interval (Sec. 4.4); negative favors the upgrade, and an interval that excludes zero marks the step as significant. The two axes share their anchor $U(m_2) = U(M2R1)$. All values are computed on the full-sweep panels and are conditional on the shared SR top-20 candidate set C (Sec. 4.4); on Defects4J, $U(M2R3)$ covers 615 of the 616 panel bugs, and the reasoning axis’s third step is read on those bugs.

Axis	Bench.	MAR at step 1/2/3			CI ₉₅ step 1→2		CI ₉₅ step 2→3
Reasoning $U(M2R\bullet)$	D4J	2.426	2.303	2.107	-0.124 [-0.281, +0.040]	-0.196 [-0.356, -0.036]	
	BIP	1.613	1.594	1.573	-0.020 [-0.139, +0.107]	-0.021 [-0.139, +0.092]	
Tier $U(M\bullet R1)$	D4J	3.963	2.426	2.054	-1.537 [-1.826, -1.254]	-0.372 [-0.538, -0.210]	
	BIP	2.757	1.613	1.490	-1.143 [-1.432, -0.870]	-0.123 [-0.266, +0.010]	

step — one level of reasoning effort versus one model tier — charges for the movement those questions measured. Table 6 gives both readings. Its uniform columns price the whole-agent steps of Table 5: the ratio of mean cost per bug between consecutive uniform chains, and the compound ratio across each full axis. Its per-role columns price the same steps taken by one slot at a time: within each of the three uniform backgrounds $U(m_i)$, each role’s sweep steps that role alone along the family $M2R1 \rightarrow M2R2 \rightarrow M2R3$ for reasoning, $M1R1 \rightarrow M2R1 \rightarrow M3R1$ for tier) while the other two slots stay put, and the table reports the median, over those nine (role, background) sweeps, of the whole chain’s cost multiplier.

Read per role, the two kinds of step live in different price bands. A single role stepping up one reasoning level multiplies the chain’s cost by a median of $\times 1.02$ ($R1 \rightarrow R2$) to $\times 1.19$ ($R2 \rightarrow R3$) on Defects4J, and $\times 1.02$ to $\times 1.17$ on BugsInPy — close enough to free that the first step is barely measurable. A single role stepping up one model tier costs a median of $\times 1.36$ ($m_1 \rightarrow m_2$) to $\times 2.07$ ($m_2 \rightarrow m_3$) on

Defects4J and $\times 1.35$ to $\times 2.03$ on BugsInPy: even at the median, a tier step charges more than the most expensive effort step.

The per-role medians must not be mistaken for the whole agent’s bill. When every slot takes the reasoning step together, the axis compounds: the uniform reasoning walk $U(M2R1) \rightarrow U(M2R3)$ multiplies mean cost per bug by $\times 2.11$ on Defects4J and $\times 2.03$ on BugsInPy — roughly doubling the chain’s cost, where the per-role medians would suggest pocket change. Nearly all of it sits on the second step: $R1 \rightarrow R2$ costs $\times 1.01$ / $\times 1.04$, while $R2 \rightarrow R3$ alone costs $\times 2.08$ / $\times 1.95$. The tier axis compounds far harder still: $U(m_1) \rightarrow U(m_3)$ multiplies cost by $\times 25.53$ (Defects4J, from \$0.0009 to \$0.0235 per bug) and $\times 21.85$ (BugsInPy, \$0.0013 to \$0.0286), with both of its steps expensive ($\times 6.91$ then $\times 3.69$ on Defects4J; $\times 5.16$ then $\times 4.24$ on BugsInPy).

Set against RQ3 and RQ4, the price list lands where the accuracy did. The only reasoning step that bought significant accuracy — Defects4J’s $R2 \rightarrow R3$ (-0.196 [-0.356, -0.036], Sec. 5.3) — is exactly the

Table 6: What a step costs (RQ5). Uniform columns: the multiplier on mean cost per bug between consecutive uniform chains along each axis of Figure 3, and the compound multiplier across the full axis. Per-role columns: the median, over the nine (role, background) sweeps $X(m_i, \cdot)$, of the whole chain’s cost multiplier when a single role takes the same step. All costs are mean USD per bug on the full-sweep panels, conditional on the shared SR top-20 candidate set C (Sec. 4.4); on Defects4J, $U(M2R3)$ covers 615 of the 616 panel bugs (Sec. 4.5).

Axis	Bench.	Uniform multiplier			Per-role median	
		step 1→2	step 2→3	compound	step 1→2	step 2→3
Reasoning	D4J	×1.01	×2.08	×2.11	×1.02	×1.19
	BIP	×1.04	×1.95	×2.03	×1.02	×1.17
Tier	D4J	×6.91	×3.69	×25.53	×1.36	×2.07
	BIP	×5.16	×4.24	×21.85	×1.35	×2.03

step carrying essentially the axis’s whole ×2.11 premium; the near-free R1→R2 step bought nothing measurable on either benchmark. And the tier axis, which concentrates the accuracy of the uniform plane (Sec. 5.4), also concentrates its cost: its cheaper-looking per-step multipliers compound to more than an order of magnitude across the axis. Whole-agent upgrades therefore pay full freight on both axes — which is what makes the per-role price band relevant: RQ1 showed that one role carries most of the accuracy, and Table 6 shows that upgrading one role costs a small multiple where upgrading the whole agent costs a large one. RQ6 asks whether chains built on that asymmetry beat the uniform frontier.

Answer to RQ5

Per role, an effort step is nearly free and a tier step is not: median whole-chain multipliers of ×1.02–×1.19 (Defects4J) and ×1.02–×1.17 (BugsInPy) per reasoning step, versus ×1.36–×2.07 and ×1.35–×2.03 per tier step. For the whole agent, however, the uniform reasoning axis compounds to ×2.11 / ×2.03, almost entirely on the R2→R3 step (×2.08 / ×1.95) — the one step whose accuracy gain RQ3 could confirm, and only on Defects4J — while the uniform tier axis compounds to ×25.53 / ×21.85.

5.6 RQ6: Do Mixed Chains Beat the Uniform Frontier?

The preceding questions located accuracy in the FINISHER (RQ1), cost in the TOOL CALLER (RQ2), and showed that whole-agent upgrades pay for all three slots at once where a single-role step costs a small multiple (RQ5). RQ6 asks whether chains built on that asymmetry actually beat the homogeneous menu. Figure 4 places every configuration in the cost/MAR plane: the 39 grid configurations plus the two off-grid uniforms of Sec. 4.5. The homogeneous menu spans the *uniform frontier* — the piecewise-linear polyline through the five uniform chains in cost order, the set of (cost, MAR) trades a practitioner restricted to a single model assignment could realize, interpolation included. A mixed configuration strictly below this polyline offers a trade the menu cannot; but a hairline crossing should not count, so the decisive notion shrinks the frontier first.

The α -shrunk frontier scales every uniform vertex radially toward the origin by a factor $(1 - \alpha)$ — on both axes, i.e. it grants the homogeneous menu a simultaneous α discount in price and α improvement in MAR. A *crosser* is a mixed configuration that still beats the frontier shrunk at $\alpha = 5.0\%$: it lies strictly below the shrunk polyline at a cost within its range, beating the discounted menu on both axes at once. Each crosser’s margin is $\alpha_{\max}$, the largest shrink it still crosses (crossing is monotone in α , so bisection finds it) — a geometric margin in units of relative discount, not a statistical statement; the statistics follow below.

By this test the answer is yes, on both benchmarks: 6 mixed configurations cross on Defects4J and 7 on BugsInPy (Table 7; the largest markers of Fig. 4), and the two sets agree — 6 of the Defects4J crossers recur on BugsInPy, which adds one more (M2R1–M2R1–M3R1). Both benchmarks are led by the same chain, M1R1–M1R1–M2R1: nano PLANNER and TOOL CALLER, mini FINISHER. Its margin is $\alpha_{\max} = 0.271$ on Defects4J and 0.281 on BugsInPy — on both benchmarks it would still cross even if every homogeneous chain were simultaneously 27.1% cheaper and 27.1% more accurate. Against its reference point on the frontier, it holds 96.3% of the frontier’s MAR at 36.4% of its cost on BugsInPy, and 95.9% at 30.2% on Defects4J. The crossers are not scattered: with the single exception of M2R1–M2R2–M2R1 (a cheap TOOL CALLER-effort bump), every one is a chain that keeps the PLANNER and TOOL CALLER at the two cheapest assignments and spends only on the FINISHER — either $U(m_1)$ with the FINISHER upgraded (M1R1–M1R1–*), or a uniform with the TOOL CALLER downgraded to nano (M2R1–M1R1–M2R1), or $U(m_2)$ with the FINISHER raised to the top tier (M2R1–M2R1–M3R1). This is exactly the shape RQ1, RQ2, and RQ5 predicted: the uniform menu is inefficient because a homogeneous agent buys the TOOL CALLER’s bulk tokens at the same tier it buys the FINISHER’s judgment.

$\alpha_{\max}$ is geometry over panel means; it does not say whether a crosser’s per-bug rank differences would survive resampling. We therefore run, for each crosser, the paired per-bug bootstrap of Sec. 4.4 against two anchors from the uniform menu: the uniform nearest to it in mean cost (does it buy real accuracy at like-for-like spend?) and the uniform nearest in MAR (does it match that accuracy at lower spend?). The right columns of Table 7 report both. Against the nearest-cost uniform, 5 of the 6 Defects4J crossers and 5 of the 7 BugsInPy crossers have intervals entirely below zero; in the cheap corner the gains are large — the leader beats $U(m_1)$ by a mean Δ AR of -1.332 [-1.576, -1.083] on Defects4J and -1.024 [-1.304, -0.763] on BugsInPy. Against the nearest-MAR uniform, most intervals straddle zero: statistically indistinguishable accuracy at a fraction of the spend, which is the crossers’ actual value proposition. The exceptions run against the crossers and should be stated plainly: on Defects4J, 2 crossers — the leader among them — are measurably worse than $U(m_2)$ (leader: +0.205 [+0.029, +0.363]), so there the leader’s crossing is bought by its cost advantage (Cost/ref 30.2%), not by matching $U(m_2)$ ’s accuracy outright; on BugsInPy no crosser is significantly worse than its nearest-MAR uniform, though the leader’s interval touches zero (+0.119 [-0.000, +0.240]).

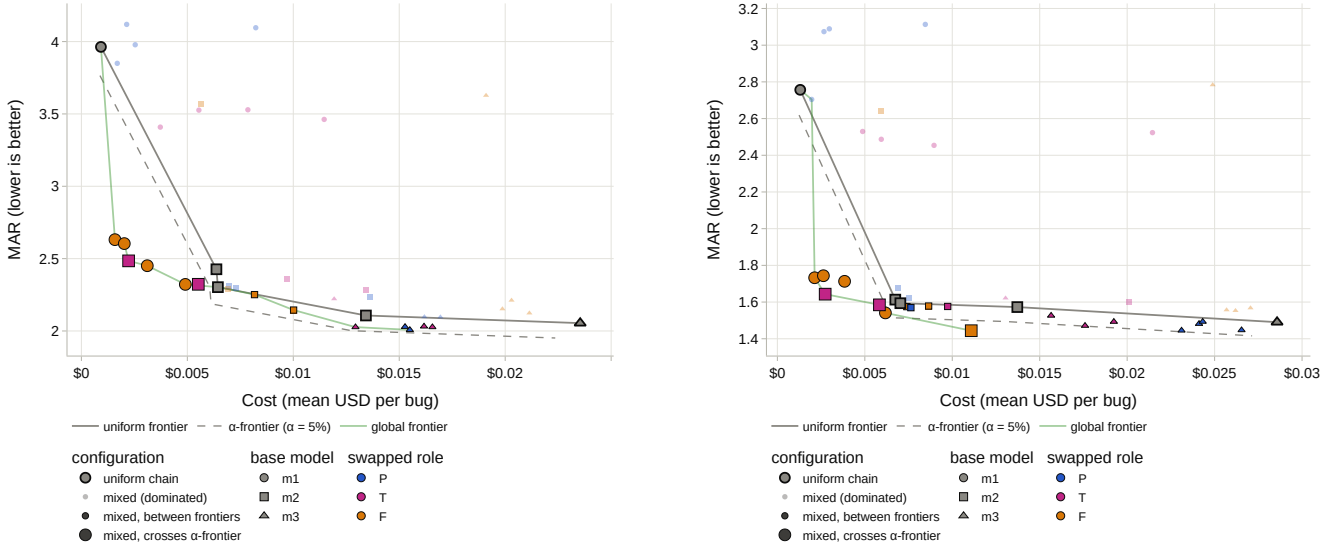


Figure 4: Mixed configurations against the uniform frontier (RQ6) on Defects4J (left, 616 panel bugs) and BugsInPy (right, 288). The grey solid line is the uniform frontier through the five uniform chains in cost order; the grey dashed line is the same frontier shrunk by $\alpha = 5.0\%$ toward the origin; the green line is the global frontier, the Pareto front over every configuration. Each configuration carries three encodings: its marker *shape* is the uniform background $U(m_i)$ it sits on, its *color* is the role swapped away from that background (grey = none, i.e. a uniform chain), and its *size* is its position relative to the two grey lines — the largest markers are the crossers of Table 7, the mid-sized ones sit between the lines, and the small faded ones are dominated. Lower left is better. MAR and mean cost per bug are computed on the full-sweep panels and are conditional on the shared SR top-20 candidate set C (Sec. 4.4); on Defects4J the off-grid uniform $U(M2R3)$ covers 615 of the 616 panel bugs.

Table 7: The crossers (RQ6): mixed configurations beating the $\alpha = 5.0\%$ -shrunk uniform frontier on both axes, ranked by $\alpha_{\max}$, the largest shrink each still crosses. MAR/ref and Cost/ref are relative to the crosser’s nearest point on the continuous (unshrunk) uniform frontier. The right columns give the paired per-bug bootstrap (Sec. 4.4) of the crosser’s AR against the uniform chain nearest to it in cost and the one nearest in MAR; negative = the crosser is more accurate. All values are computed on the full-sweep panels and are conditional on the shared SR top-20 candidate set C ; on Defects4J the uniform $U(M2R3)$ covers 615 of the 616 panel bugs.

Bench.	Config	MAR	\$/bug	MAR/ref	Cost/ref	$\alpha_{\max}$	vs. nearest-cost uniform		vs. nearest-MAR uniform	
							ref	$\Delta AR [CI_{95}]$	ref	$\Delta AR [CI_{95}]$
D4J	M1R1-M1R1-M2R1	2.631	0.0016	0.959	0.302	0.271	$U(m_1)$	-1.33 [-1.58, -1.08]	$U(m_2)$	+0.20 [+0.03, +0.36]
	M2R1-M1R1-M2R1	2.485	0.0022	0.958	0.386	0.263	$U(m_1)$	-1.48 [-1.75, -1.20]	$U(m_2)$	+0.06 [-0.08, +0.20]
	M1R1-M1R1-M2R2	2.604	0.0020	0.962	0.377	0.248	$U(m_1)$	-1.36 [-1.61, -1.11]	$U(m_2)$	+0.18 [+0.01, +0.35]
	M1R1-M1R1-M2R3	2.451	0.0031	0.965	0.521	0.212	$U(m_1)$	-1.51 [-1.78, -1.25]	$U(m_2)$	+0.02 [-0.14, +0.19]
	M1R1-M1R1-M3R1	2.322	0.0049	0.997	0.763	0.123	$U(m_2)$	-0.10 [-0.26, +0.05]	$U(M2R2)$	+0.02 [-0.13, +0.17]
	M2R1-M2R2-M2R1	2.323	0.0055	0.998	0.858	0.082	$U(m_2)$	-0.10 [-0.20, -0.01]	$U(M2R2)$	+0.02 [-0.12, +0.15]
BIP	M1R1-M1R1-M2R1	1.732	0.0021	0.963	0.364	0.281	$U(m_1)$	-1.02 [-1.30, -0.76]	$U(m_2)$	+0.12 [-0.00, +0.24]
	M2R1-M1R1-M2R1	1.643	0.0027	0.963	0.434	0.268	$U(m_1)$	-1.11 [-1.41, -0.83]	$U(m_2)$	+0.03 [-0.04, +0.10]
	M1R1-M1R1-M2R2	1.744	0.0026	0.968	0.450	0.242	$U(m_1)$	-1.01 [-1.30, -0.76]	$U(m_2)$	+0.13 [-0.00, +0.26]
	M1R1-M1R1-M2R3	1.713	0.0038	0.977	0.632	0.168	$U(m_1)$	-1.04 [-1.36, -0.76]	$U(m_2)$	+0.10 [-0.04, +0.24]
	M2R1-M2R1-M3R1	1.444	0.0111	0.914	0.990	0.085	$U(M2R3)$	-0.13 [-0.25, -0.02]	$U(m_3)$	-0.05 [-0.13, +0.03]
	M2R1-M2R2-M2R1	1.585	0.0058	0.982	0.866	0.072	$U(m_2)$	-0.03 [-0.15, +0.07]	$U(M2R2)$	-0.01 [-0.15, +0.12]
	M1R1-M1R1-M3R1	1.541	0.0062	0.967	0.878	0.063	$U(m_2)$	-0.07 [-0.18, +0.02]	$U(M2R3)$	-0.03 [-0.12, +0.06]

Answer to RQ6

Yes: 6 (Defects4J) and 7 (BugsInPy) mixed configurations beat the uniform frontier even after it is shrunk by 5.0% on both axes, and all but one follow the frugal-PLANNER/TOOL CALLER, stronger-FINISHER shape that RQ1–RQ5 predicted. Both benchmarks are led by M1R1–M1R1–M2R1 ($\alpha_{\max} = 0.271 / 0.281$), which on BugsInPy holds 96.3% of the frontier’s MAR at 36.4% of its cost. The paired per-bug bootstrap confirms the cheap-corner crossers beat their like-cost uniform outright (leader vs. $U(m_1)$: $-1.332 [-1.576, -1.083] / -1.024 [-1.304, -0.763]$), while against the like-accuracy uniform $U(m_2)$ the leader is measurably slightly worse on Defects4J ($+0.205 [+0.029, +0.363]$) and borderline on BugsInPy ($+0.119 [-0.000, +0.240]$) — its case rests on price, not parity.

Cross-language consistency. Every verdict in this section was read twice, once per benchmark, and the agreement is not limited to the headline conclusions. Over the 39 grid configurations shared by both benchmarks, the Spearman rank correlation between Defects4J and BugsInPy is $\rho = 0.94$ on MAR ($p = 7 \times 10^{-19}$) and $\rho = 0.98$ on mean cost per bug ($p = 3 \times 10^{-29}$): the grid’s *ordering* — which assignments help, which merely spend — transfers almost intact across language, benchmark, and coverage tooling, from Java under GZoltar to Python under FauxPy. The two benchmarks differ in level (BugsInPy’s panel is easier in absolute MAR) but hardly in structure. What that agreement implies beyond these two ecosystems is, of course, not something two benchmarks can settle.

6 Conclusion

Which parts of an LLM fault-localization agent need an expensive model, and which only look like they do? We reimplemented FlexFL’s two-stage pipeline, split its refinement agent Agent4LR into three role-bound slots — PLANNER, TOOL CALLER, FINISHER — and assigned each slot a model tier and reasoning effort independently, evaluating the resulting grid of 39 configurations on 904 bugs (616 Defects4J, 288 BugsInPy) with per-call token and USD accounting, under prefix-sharing execution that turns every single-role swap into a paired comparison. All findings are conditional on the shared SR top-20 candidate set C and the full-sweep panels (Sec. 4.4).

The answer is that the agent’s accuracy and its cost live in different slots. The FINISHER decides accuracy: upgrading it alone closes 86.0% (Defects4J) and 96.0% (BugsInPy) of the all-nano-to-all-gpt-5 MAR gap. The TOOL CALLER dominates spend, and the PLANNER’s assignment moves both axes least. Whole-agent upgrades pay for all three slots at once — the uniform tier axis compounds to $\times 25.53$ on Defects4J and $\times 21.85$ on BugsInPy — and mixed chains exploit exactly that asymmetry: 6 (Defects4J) and 7 (BugsInPy) mixed configurations beat the uniform frontier even after it is shrunk by 5.0% on both axes, led on both benchmarks by M1R1–M1R1–M2R1, which holds 95.9% / 96.3% of its frontier reference’s MAR at 30.2% / 36.4% of its cost. The practical recipe is correspondingly short: keep the PLANNER and TOOL CALLER at the cheapest tier, spend on the FINISHER alone, and buy whole-agent reasoning effort only where a specific step has shown a measurable return (Sec. 5.3).

Two follow-ups suggest themselves. First, repetition with cheap chains: every configuration here ran once, and at the leader’s price several independent passes still cost less than one uniform run of comparable accuracy, so whether aggregating repeated cheap runs closes the leader’s residual accuracy gap to the uniform chains is an experiment our data prices but does not run. Second, extending the role decomposition upstream: SR was held fixed as a single llama3.1:8b stage shared by every configuration, and the same swap-and-account method applied to Agent4SR and the classical rankers would complete the per-role cost picture of the pipeline end to end.

References

- [1] Rui Abreu, Peter Zoetewij, and Arjan J. C. van Gemund. 2006. An Evaluation of Similarity Coefficients for Software Fault Localization. In *12th Pacific Rim International Symposium on Dependable Computing (PRDC’06)*. IEEE.
- [2] Rui Abreu, Peter Zoetewij, and Arjan J. C. van Gemund. 2009. Spectrum-Based Multiple Fault Localization. In *2009 IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE. doi:10.1109/ASE.2009.25
- [3] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.
- [4] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. *Proceedings of the ACM on Software Engineering* 1, FSE (2024). arXiv:2308.05487 [cs.SE]
- [5] Yiling Lou, Qihao Zhu, Jinhao Dong, Xia Li, Zeyu Sun, Dan Hao, Lu Zhang, and Lingming Zhang. 2021. Boosting coverage-based fault localization via graph-based representation learning. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 664–676.
- [6] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2024. AgentFL: Scaling LLM-based Fault Localization to Project-Level Context. arXiv:2403.16362 [cs.SE]
- [7] Abdul Razzaq, Jim Buckley, James Vincent Patten, Muslim Chochlov, and Ashish Rajendra Sai. 2021. BoostNSift: A Query Boosting and Code Sifting Technique for Method Level Bug Localization. arXiv:2108.12901 [cs.SE]
- [8] Yonghao Wu, Zheng Li, Jie M Zhang, Mike Papadakis, Mark Harman, and Yong Liu. 2023. Large language models in fault localisation. *arXiv preprint arXiv:2308.15276* (2023).
- [9] Chuyang Xu, Zhongxin Liu, Xiaoxue Ren, Gehao Zhang, Ming Liang, and David Lo. 2024. FlexFL: Flexible and Effective Fault Localization with Open-Source Large Language Models. arXiv:2411.10714 [cs.SE]